

OPERATING SYSTEMS (UE24CS242B)

Unit 1 – Process Concept (Process Management)

Process Concept

An operating system executes a variety of programs:

- **Batch systems** → execute *jobs*
- **Time-shared systems** → execute *user programs* or *tasks*

In operating-system literature, the terms **job** and **process** are often used interchangeably.

Process vs Program

- A **program** is a **passive entity**
 - Stored on disk as an executable file
- A **process** is an **active entity**
 - A program in execution

A program becomes a process when:

- Its executable file is loaded into memory
- Execution begins (via GUI click, command line, script, etc.)

One program can have multiple processes

- Example: Multiple users running the same editor simultaneously

Process Concept – Memory Representation

A process consists of several parts in memory:

1. Text Section

- Program code
- Instructions to be executed

2. Current Activity

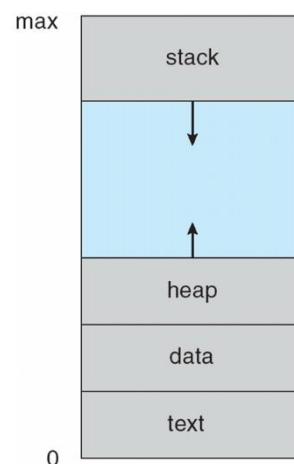
- Program Counter
- Processor Registers

3. Stack

- Temporary data
- Function parameters
- Return addresses
- Local variables

4. Data Section

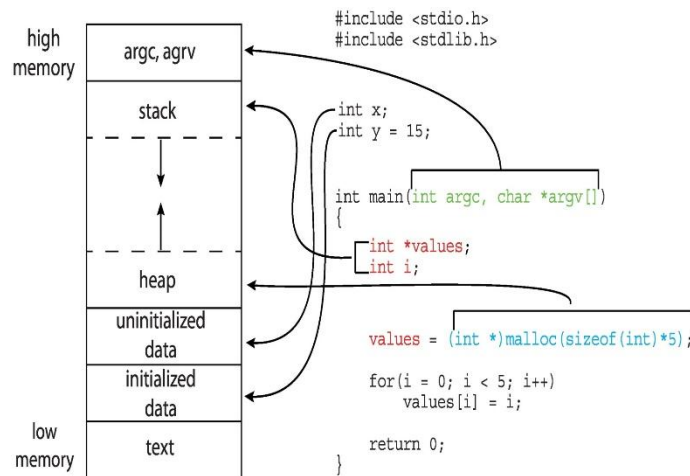
- Global and static variables



5. Heap

- Dynamically allocated memory during runtime

Memory Layout of a C Program



This layout explains how different segments of a program are organized in memory during execution.

Memory Layout of a Process

```
suresh@PES1UG19CS553:~/OS$ gcc -o q1b.exe q1b.c
suresh@PES1UG19CS553:~/OS$ ./q1b.exe 5
Process id: 23059
```

```
suresh@PES1UG19CS553:~$ pmap 23059
23059:  ./q1b.exe 5
000055cefc914000    4K r-x-- q1b.exe
000055cefc914000    4K r---- q1b.exe
000055cefc915000    4K rw--- q1b.exe
000055cefe6e7000   132K rw--- [ anon ]
00007f26ef253000   1948K r-x-- libc-2.27.so
00007f26ef43a000   2048K ----- libc-2.27.so
00007f26ef63a000    16K r---- libc-2.27.so
00007f26ef63e000     8K rw--- libc-2.27.so
00007f26ef640000    16K rw--- [ anon ]
00007f26ef644000   164K r-x-- ld-2.27.so
00007f26ef857000     8K rw--- [ anon ]
00007f26ef86d000     4K r---- ld-2.27.so
00007f26ef86e000     4K rw--- ld-2.27.so
00007f26ef86f000     4K rw--- [ anon ]
00007ffda2b15000   136K rw--- [ stack ]
00007ffda2bae000    12K r---- [ anon ]
00007ffda2bb1000     8K r-x-- [ anon ]
fffffffff6000000    4K --x-- [ anon ]
total             4524K
```

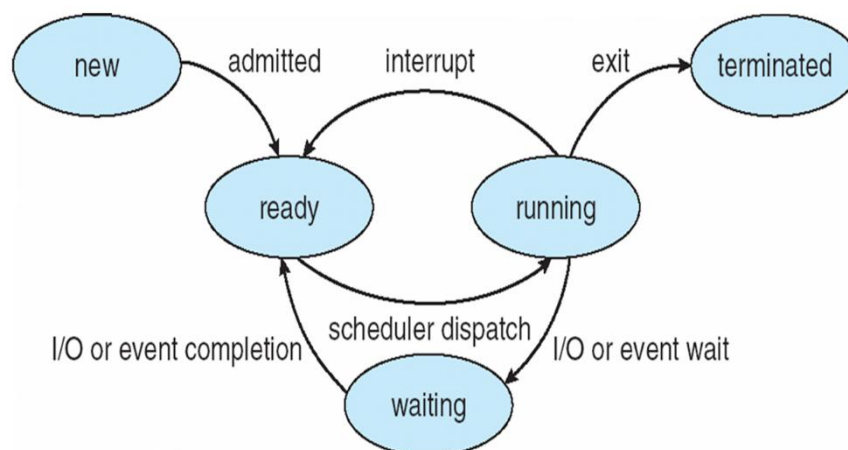
```
suresh@PES1UG19CS553:~$ cat /proc/23059/maps
55cefc914000-55cefc915000 r-xp 00000000 08:07 4338399 /home/suresh/OS/q1b.exe
55cefc914000-55cefc915000 r--p 00000000 08:07 4338399 /home/suresh/OS/q1b.exe
55cefc915000-55cefc916000 rw-p 00001000 08:07 4338399 /home/suresh/OS/q1b.exe
55cefe6e7000-55cefe708000 rw-p 00000000 00:00 0 [heap]
7f26ef253000-7f26ef43a000 r-xp 00000000 08:07 526643 /lib/x86_64-linux-gnu/libc-2.27.so
7f26ef43a000-7f26ef63a000 ---p 001e7000 08:07 526643 /lib/x86_64-linux-gnu/libc-2.27.so
7f26ef63a000-7f26ef63e000 r--p 001e7000 08:07 526643 /lib/x86_64-linux-gnu/libc-2.27.so
7f26ef63e000-7f26ef640000 rw-p 001eb000 08:07 526643 /lib/x86_64-linux-gnu/libc-2.27.so
7f26ef640000-7f26ef644000 rw-p 00000000 00:00 0
7f26ef644000-7f26ef66d000 r-xp 00000000 08:07 526515 /lib/x86_64-linux-gnu/ld-2.27.so
7f26ef66d000-7f26ef859000 rw-p 00000000 00:00 0
7f26ef86d000-7f26ef86e000 r--p 00029000 08:07 526515 /lib/x86_64-linux-gnu/ld-2.27.so
7f26ef86e000-7f26ef86f000 rw-p 0002a000 08:07 526515 /lib/x86_64-linux-gnu/ld-2.27.so
7f26ef86f000-7f26ef870000 rw-p 00000000 00:00 0
7ffda2b15000-7ffda2b37000 rw-p 00000000 00:00 0
7ffda2bae000-7ffda2bb1000 r--p 00000000 00:00 0
7ffda2bb1000-7ffda2bb3000 r-xp 00000000 00:00 0
fffffffff6000000-fffffffff601000 --xp 00000000 00:00 0 [vsyscall]
suresh@PES1UG19CS553:~$
```

Process State

As a process executes, it changes state.

Common Process States

1. **New**
 - Process is being created
2. **Running**
 - Instructions are currently being executed
3. **Waiting**
 - Process is waiting for an event (e.g., I/O completion)
4. **Ready**
 - Process is waiting to be assigned to a CPU
5. **Terminated**
 - Process has finished execution



Process Control Block (PCB)

Each process is represented in the operating system by a **Process Control Block (PCB)** (also called **Task Control Block**).

Contents of Process Control Block

The PCB contains all information needed to manage a process:

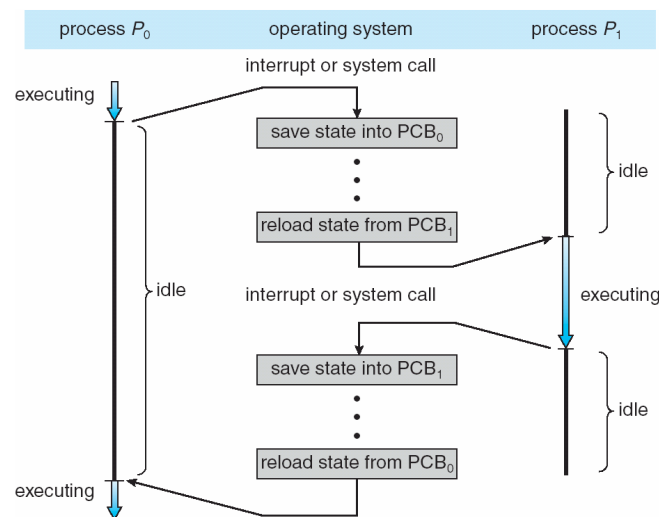
- **Process state**
 - New, ready, running, waiting, terminated
- **Program counter**
 - Address of next instruction to execute
- **CPU registers**
 - Contents of all process-specific registers
- **CPU scheduling information**

- Priority
- Scheduling queue pointers
- **Memory-management information**
 - Memory allocated to the process
- **Accounting information**
 - CPU time used
 - Elapsed time
 - Time limits
- **I/O status information**
 - I/O devices allocated
 - List of open files

process state
process number
program counter
registers
memory limits
list of open files
...

CPU Switch from Process to Process

When the CPU changes from executing one process to another, the system performs a **context switch**.



Process Scheduling

The objective of process scheduling is to:

- Maximize CPU utilization
- Support time sharing
- Ensure fairness among processes

A **process scheduler** selects a process for execution on the CPU.

Scheduling Queues

The operating system maintains several scheduling queues:

1. **Job Queue**

- All processes in the system

2. Ready Queue

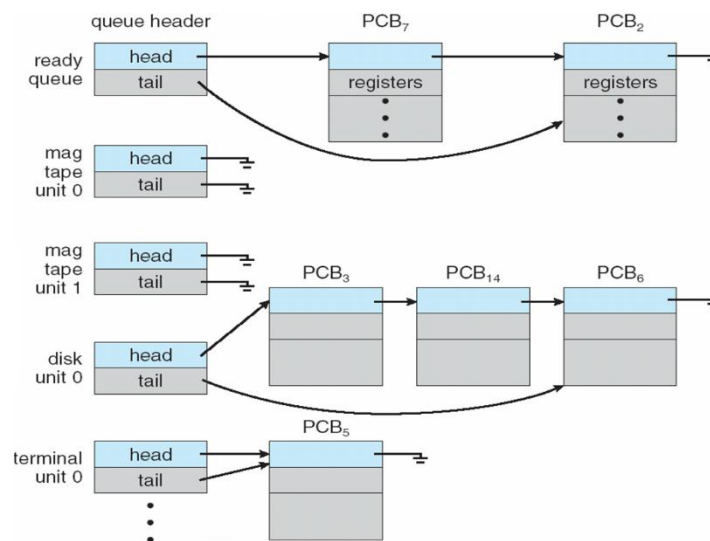
- Processes in main memory
- Ready to execute

3. Device Queues

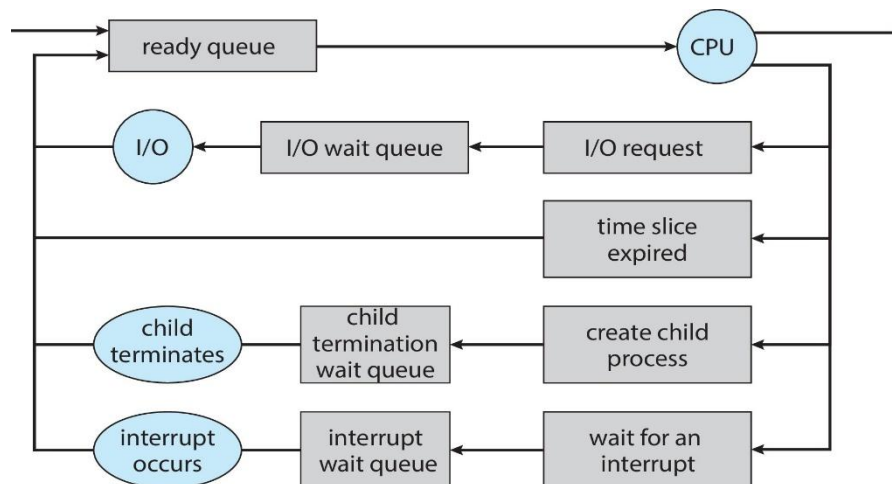
- Processes waiting for I/O devices

Processes migrate among these queues during execution.

Ready Queue and I/O Device Queues



Representation of Process Scheduling



Schedulers

Short-Term Scheduler (CPU Scheduler)

- Selects the next process to execute on the CPU
- Invoked very frequently (milliseconds)
- Must be extremely fast
- Often the only scheduler in a system

Long-Term Scheduler (Job Scheduler)

- Selects processes to be admitted into the ready queue
- Invoked infrequently (seconds or minutes)
- Controls the **degree of multiprogramming**
- May be slow

Types of Processes

Processes are broadly classified as:

1. I/O-Bound Process

- Spends more time doing I/O
- Many short CPU bursts

2. CPU-Bound Process

- Spends more time doing computation
- Few long CPU bursts

The long-term scheduler tries to maintain a **good mix** of both.

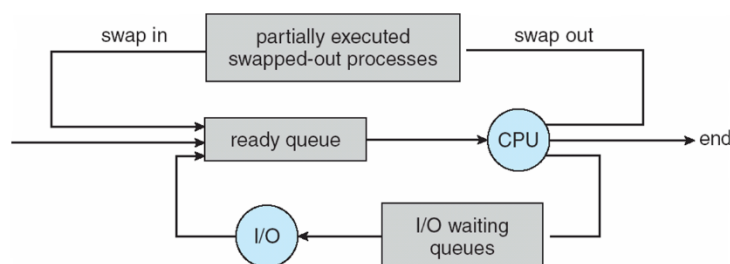
Medium-Term Scheduler

A **medium-term scheduler** may be introduced to:

- Reduce the degree of multiprogramming

Swapping

- Process is removed from memory
- Stored on disk
- Later brought back to continue execution



Context Switch

A **context switch** occurs when:

- CPU switches from one process to another

Steps involved:

1. Save the state of the current process in its PCB
2. Load the saved state of the next process

Key Points

- Context switch time is **overhead**
- No useful work is done during switching
- Time depends on:
 - Hardware support
 - PCB complexity
- Some CPUs support multiple register sets to reduce switch time

Operations on Processes

The operating system must provide mechanisms for:

- **Process creation**
- **Process termination**

Process Creation

- A **parent process** can create **child processes**
- Child processes may create further processes
- Forms a **process tree**

Each process is identified by a **Process ID (PID)**.

Resource Sharing Options

- Parent and child share all resources
- Child shares a subset of parent's resources
- Parent and child share no resources

Execution Options

- Parent and child execute concurrently
- Parent waits until child terminates

Process Creation using fork()

- fork() creates a new process
- Child process is a duplicate of the parent
- Address space is copied

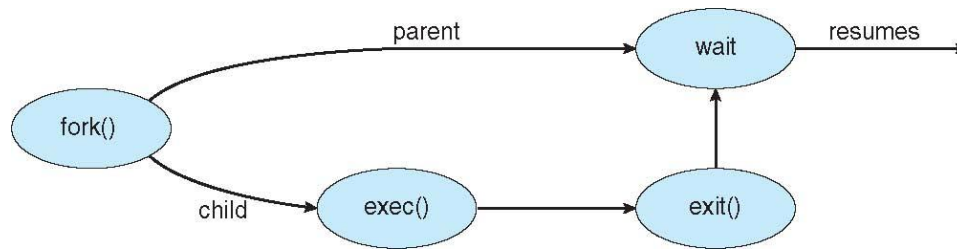
To run a different program:

- exec() is used
- Replaces the process memory with a new program

UNIX Example

- fork() → creates child process
- exec() → loads new program into child

C Program – Forking Separate Process



Process Termination

A process terminates when:

- It executes its last statement
- Calls the `exit()` system call

Termination Details

- Status value returned from child to parent
- Parent retrieves status using `wait()`
- OS deallocates all process resources

Parent Terminating Child

Parent may terminate a child using `abort()` if:

- Child exceeds allocated resources
- Task assigned to child is no longer needed
- Parent itself is terminating

Cascading Termination

Some operating systems do not allow a child to exist without a parent.

- If parent terminates → all children terminate
- This is called **cascading termination**
- Initiated by the operating system

Zombie and Orphan Processes

- **Zombie process**
 - Child terminates
 - Parent has not called `wait()`
- **Orphan process**
 - Parent terminates before child
 - Child is adopted by `init/system` process

Important Scenarios

- What happens if parent terminates before child?

- What happens if child terminates before parent (parent sleeping)?
- How to determine the state of a process?

These are handled using process tables and system calls like wait().